utils Â¶

Utilities for plan handling.

LOGGER `module-attribute` Â¶

```
LOGGER = getLogger(__name__)
```

FeatureUID Â¶

Bases: `NamedTuple`

Tuple containing a unique representation of a feature based on the feature and operator names.

name `instance-attribute` Â¶

```
name: str
```

op_name `instance-attribute` Â¶

```
op_name: Optional[str]
```

removed `class-attribute` `instance-attribute` Â¶

```
removed: bool = False
```

is_field_removed Â¶

```
is_field_removed() -> bool
```

Returns True if the FeatureUID is created from a field being removed in a MapOperator.

is_schema_field Â¶

```
is_schema_field() -> bool
```

True if the FeatureUID represents a schema field.

HtmlDiff Â¶

diff_dict `staticmethod` Â¶

```
diff_dict(d1: Dict, d2: Dict) -> Dict
```

Finds the dictionaries with items that contain different values.

diff_dicts_html Â¶

```
diff_dicts_html(dicts_1: List[Dict], dicts_2: List[Dict], join_keys: List[str], headers:
List[str] | None = None, only_diff: bool = True) -> str
```

Finds the differences between two lists of dictionaries.

Differences can be either updates, additions or deletions and are formatted side by side in an HTML table. Updates take place if the values of 'join_keys' are equal.

Parameters:

Name	Type	Description	Default
dicts_1 Â¶	list of dict	List of dicts to be compared.	*required*
dicts_2 Â¶	list of dict	List of dicts to be compared.	*required*
join_keys Â¶	list of str	Keys to pair dictionaries in both lists and evaluate equality.	*required*
headers Â¶	list, optional, by default None	Headers to be displayed on the diff table, length 2.	None
only_diff Â¶	bool, by default True.	Report only dictionaries with changes.	True

Returns:

Type	Description
str	Differences in HTML formatted code.

WindowInterval Â¶

```
WindowInterval(window_expression: str)
```

Manipulate window intervals/expressions. Mostly to generate metric names.

duration class-attribute instance-attribute Â¶

```
duration: Optional[int] = None
```

is_simple property Â¶

```
is_simple: bool
```

True when the window definition follows the simple format of ' '.

unit class-attribute instance-attribute Â¶

```
unit: Optional[str] = None
```

window_expression instance-attribute Â¶

```
window_expression = require_type(window_expression, str, 'window_expression')
```

get_name Â¶

```
get_name() -> Optional[str]
```

Return the normalized name of the window expression.

Expressions that do not follow the ' ' pattern do not get a name.

Returns:

Type	Description
`str`	Normalized name of the window expression.

get_normed_time_unit `staticmethod` Â¶

```
get_normed_time_unit(time_unit) -> Optional[str]
```

Given a valid time_unit it returns the normalized string.

has_simple_tokens Â¶

```
has_simple_tokens() -> bool
```

True when there are two tokens, ie, the window expression follows the simple format ' '.

has_valid_simple_duration Â¶

```
has_valid_simple_duration() -> bool
```

True when the duration of a simple window expression is valid.

has_valid_simple_unit Â¶

```
has_valid_simple_unit() -> bool
```

True when the unit of a simple window expression is valid.

tokenize Â¶

```
tokenize() -> List[str]
```

Return a sequence of tokens from the window expression.

deduplicate_names Â¶

```
deduplicate_names(names: list[str]) -> list[str]
```

Deduplicate the names in the list by appending '_n+1'.

Parameters:

Name	Type	Description	Default
`names` Â¶	`list`	List of names.	*required*

Returns:

Type	Description
`str`	Deduplicated list of names.

in_jupyter Â¶

```
in_jupyter() -> bool
```

Returns true if run from a jupyter notebook/lab.

parse_literal Â¶

```
parse_literal(val: Any | None) -> Any
```

Given a value either in a primitive type or as a literal it will return the correct type.

validate_plan_is_deployable Â¶

```
validate_plan_is_deployable(func)
```

Validates the Structure of a Plan is supported.

Raises:

Type	Description
`NotImplementedError`	If the PlanStructure type is not supported